Supervised Learning Framework and Linear Regression

In supervised learning, algorithms learn a mapping function f: X -> y from labeled training pairs (x_i, y_i). In linear regression, the model predicts target y_hat = theta_0 + theta_1 * x_1 + ... + theta_p * x_p.

Cost Function: The performance is evaluated using Mean Squared Error (MSE): J(theta) = (1 / 2m) * sum((h_theta(x_i) - y_i)^2). Because MSE is a convex quadratic function, it possesses a single global minimum.

Gradient Descent and Analytical Normal Equation

Gradient Descent iteratively updates model parameters opposite to the gradient: theta_j := theta_j - alpha * (d/d theta_j) J(theta), where alpha is the learning rate.
- Batch Gradient Descent uses all m examples per step (smooth convergence, computationally heavy).
- Stochastic Gradient Descent (SGD) updates parameters per individual example (fast, noisy path).
- Mini-Batch Gradient Descent strikes an optimal balance using batches of 32 to 256 samples.

Analytical Solution: The Normal Equation computes theta = (X^T * X)^(-1) * X^T * y in O(p^3) time without iterative tuning.

Overfitting, Underfitting, and Regularization (L1 vs L2)

Overfitting occurs when a complex model fits noise in training data, exhibiting low training error but high test error (High Variance). Underfitting occurs when a model is too simple to capture trends (High Bias).

Regularization penalizes large parameter magnitudes to shrink variance:
- Ridge Regression (L2): Adds lambda * sum(theta_j^2) to cost function. Shrinks parameters smoothly toward zero.
- Lasso Regression (L1): Adds lambda * sum(|theta_j|) to cost function. Drives less informative coefficients to absolute zero, performing automatic feature selection.